期中考解答（A卷，2026/10/29）

本解答以課程 lab（Ch01–Ch06 中文 lab）的寫法為主；執行環境對齊電腦教室 `packages.txt`（Python 3.9.13、sklearn 1.6.1、statsmodels 0.13.2、ISLP 0.4.0）。

題目一：使用 "Data1" 資料集進行迴歸分析

請使用 **"Data1"** 資料集，其中應變數為 y，自變數為 $V_1, V_2, \ldots, V_{10}$。請回答下列問題：

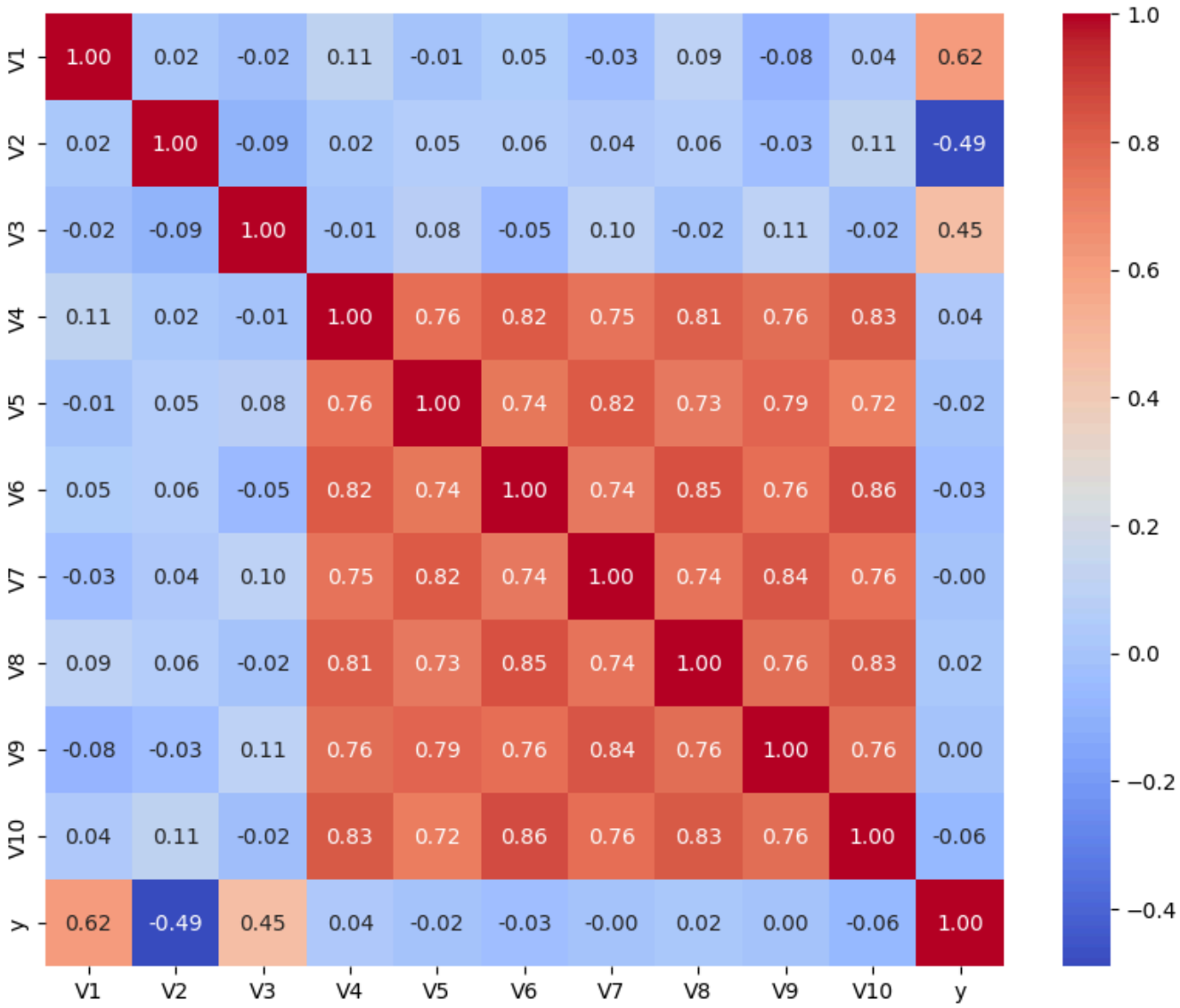
- 當使用具有隨機性的演算法（例如隨機抽樣）時，請設定 `random_state=2026` 以確保結果可重現。
- 若所選模型**不具尺度不變性（scale-invariance）**，請在建模前對輸入的自變數進行標準化（standardization）。

(a) 以 `describe()` 檢視各變數的摘要統計量，並計算所有變數間的相關係數矩陣，以熱圖（heatmap）繪製。自變數之間是否存在明顯的相關結構？哪些自變數與 y 的相關最強？ (6%)

```
df1 = pd.read_csv('Data1.csv')
df1.describe()
```

	V1	V2	V3	V4	V5	V6	V7	
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.0
mean	0.062760	0.126241	-0.158997	-0.218175	-0.341054	-0.271358	-0.314374	-0.2
std	1.112106	1.098963	1.075341	1.080636	0.945967	1.041645	1.013384	0.9
min	-2.451800	-3.056100	-3.000000	-2.727300	-3.219300	-3.370700	-2.802700	-2.8
25%	-0.614400	-0.732975	-0.831700	-0.860075	-1.004175	-1.020675	-0.995325	-1.0
50%	-0.054300	0.068800	-0.177400	-0.107850	-0.233100	-0.133600	-0.203250	-0.1
75%	0.687200	0.825800	0.695625	0.499800	0.410650	0.528950	0.388800	0.4
max	3.518100	3.000000	2.196400	2.262800	1.591800	2.000000	2.076500	2.0

```
corr = df1.corr(numeric_only=True)
fig, ax = subplots(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', ax=ax);
```




```
block = corr.loc['V4':'V10', 'V4':'V10'].values[np.triu_indices(7, 1)]
R('a_block', block.mean(), 2)
R('a_top', corr['y'].drop('y').abs().sort_values(ascending=False).index[:3].tolist())
corr['y'].drop('y').round(3)
```

```
V4–V10 兩兩相關係數平均：0.78
與 y 相關最強的三個自變數：V1, V2, V3
```

```
V1     0.620
V2    -0.490
V3     0.455
V4     0.037
V5    -0.019
V6    -0.029
V7    -0.005
V8     0.020
V9     0.004
V10   -0.062
Name: y, dtype: float64
```

> **參考答案**：V_4–V_{10} 彼此高度正相關（兩兩相關係數平均約 0.78），形成一個相關區塊，但它們與 y 的相關都很弱；與 y 相關（絕對值）最強的三個自變數是 V1, V2, V3。

(b) 使用 `train_test_split` 隨機選取四分之三的觀測資料作為訓練集（training set），其餘四分之一作為測試集（test set）（`test_size=0.25`）。使用訓練集擬合包含全部十個自變數與截距的多元線性迴歸模型。是否至少有一個自變數與 y 有關（F 檢定）？在顯著水準 0.05 下，哪些自變數顯著？ (7%)

```
X = df1.drop(columns='y')
y = df1['y']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2026)
X_train.shape, X_test.shape
```

```
((75, 10), (25, 10))
```

```
design = MS(X.columns).fit(X_train)      # MS 會自動加入 intercept
results = sm.OLS(y_train, design.transform(X_train)).fit()
print(results.summary())          # Prob (F-statistic) 即 F 檢定的 p 值
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.821
Model:                            OLS   Adj. R-squared:                  0.793
Method:                 Least Squares   F-statistic:                     29.35
Date:                Tue, 20 Oct 2026   Prob (F-statistic):           3.39e-20
Time:                        20:52:44   Log-Likelihood:                -184.29
No. Observations:                  75   AIC:                             390.6
Df Residuals:                      64   BIC:                             416.1
Df Model:                          10
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
intercept      1.4201      0.384      3.700      0.000       0.653       2.187
V1             3.6361      0.330     11.017      0.000       2.977       4.295
V2            -2.4957      0.362     -6.894      0.000      -3.219      -1.772
V3             2.5796      0.346      7.465      0.000       1.889       3.270
V4             0.0364      0.790      0.046      0.963      -1.541       1.614
V5            -0.4071      0.765     -0.532      0.596      -1.935       1.120
V6             0.2028      0.877      0.231      0.818      -1.550       1.955
V7             0.5587      0.775      0.721      0.473      -0.989       2.106
V8             0.4063      0.804      0.505      0.615      -1.200       2.012
V9            -0.2961      0.739     -0.400      0.690      -1.773       1.181
V10           -0.6727      0.846     -0.795      0.430      -2.363       1.018
==============================================================================
Omnibus:                       81.597   Durbin-Watson:                   2.063
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1151.612
Skew:                           3.080   Prob(JB):                    8.52e-251
Kurtosis:                      21.181   Cond. No.                         7.79
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
R('b_Fp', '%.2e' % results.f_pvalue)
R('b_sig', results.pvalues.drop('intercept')[lambda s: s < 0.05].index.tolist())
results.f_pvalue
```

```
F 檢定 p 值：3.39e-20
顯著的自變數 (α=0.05)：V1, V2, V3
```

```
3.3939941262363606e-20
```

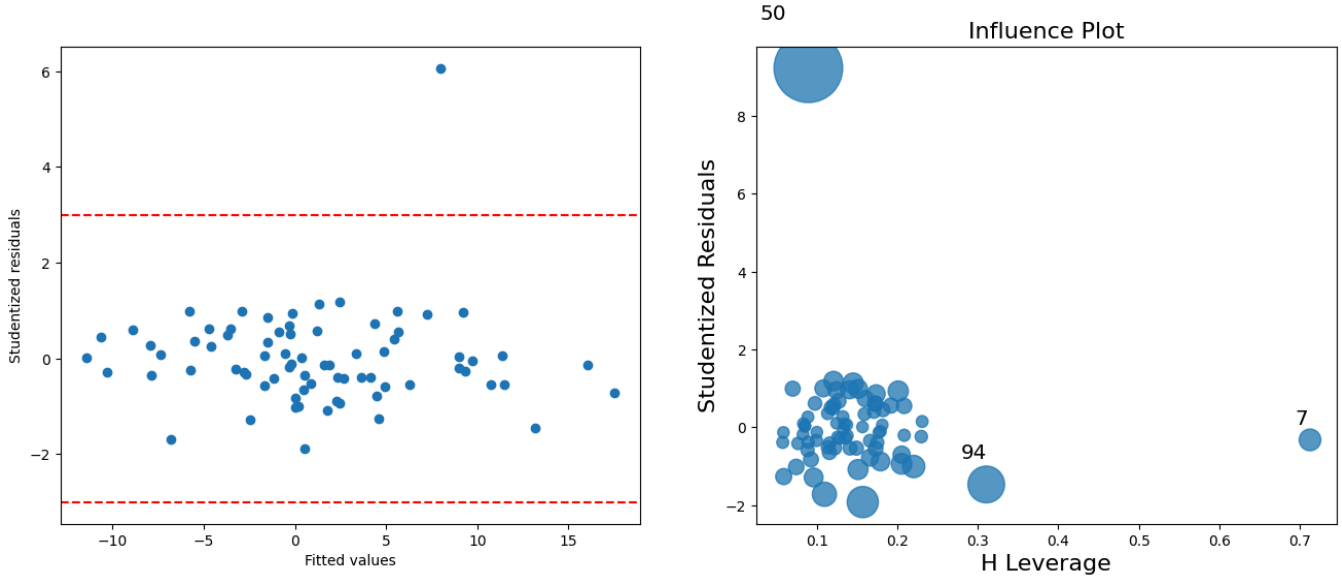
> **參考答案**：F 檢定的 p 值為 3.39e-20，至少有一個自變數與 y 有關。顯著的自變數為 V1, V2, V3。

(c) 對 (b) 的模型計算每一筆訓練資料的 studentized residual、槓桿值（leverage，h_i）與 Cook's distance（D_i），並繪製 studentized residual 對擬合值（fitted values）以及 studentized residual 對槓桿值的散佈圖。以 $|r_i| > 3$、$h_i > 3(p+1)/n$、$D_i > 4/n$ 為準則（n 為訓練集樣本數，$p = 10$），分別列出被標記的觀測值（以原始資料的列索引 index 表示）。哪一筆是「高槓桿但不是影響點（influential point）」？哪一筆是影響點？請說明理由。 (10%)

```
infl = results.get_influence()
diag = pd.DataFrame({'stud_resid': infl.resid_studentized_internal,
                     'leverage': infl.hat_matrix_diag,
                     'cooks_d': infl.cooks_distance[0]}, index=X_train.index)
n, p = X_train.shape
h_cut, d_cut = 3 * (p + 1) / n, 4 / n
print('h cut =', round(h_cut, 3), ' D cut =', round(d_cut, 3))
```

```
h cut = 0.44  D cut = 0.053
```

```
fig, axes = subplots(1, 2, figsize=(16, 6))
axes[0].scatter(results.fittedvalues, diag['stud_resid'])
axes[0].axhline(3, c='r', ls='--'); axes[0].axhline(-3, c='r', ls='--')
axes[0].set_xlabel('Fitted values'); axes[0].set_ylabel('Studentized residuals')
sm.graphics.influence_plot(results, criterion="cooks", ax=axes[1]);
```




```
flag_r = diag.index[diag['stud_resid'].abs() > 3].tolist()
flag_h = diag.index[diag['leverage'] > h_cut].tolist()
flag_d = diag.index[diag['cooks_d'] > d_cut].tolist()
print('|r|>3 :', flag_r, '\nh > cut:', flag_h, '\nD > 4/n:', flag_d)
i_lev, i_inf = diag['leverage'].idxmax(), diag['cooks_d'].idxmax()
R('c_r', flag_r); R('c_h', flag_h); R('c_d', flag_d)
R('c_hcut', h_cut); R('c_dcut', d_cut)
R('c_lev', i_lev); R('c_lev_h', diag.loc[i_lev, 'leverage']); R('c_lev_r', diag.loc[i_lev,
```

```
'stud_resid'], 2); R('c_lev_d', diag.loc[i_lev, 'cooks_d'])
R('c_inf', i_inf); R('c_inf_r', diag.loc[i_inf, 'stud_resid'], 2); R('c_inf_h', diag.loc[i_i
nf, 'leverage']); R('c_inf_d', diag.loc[i_inf, 'cooks_d'])
diag.loc[sorted(set(flag_r + flag_h + flag_d))].round(3)
```

```
|r|>3 : [50]
h > cut: [7]
D > 4/n: [50, 9, 94]
|r|>3 的觀測值：50
高槓桿 h>3(p+1)/n 的觀測值：7
Cook's D>4/n 的觀測值：50, 9, 94
槓桿門檻 3(p+1)/n：0.440
Cook's D 門檻 4/n：0.053
槓桿最大的觀測值：7
  其槓桿值：0.713
  其 studentized residual：-0.33
  其 Cook's D：0.024
Cook's D 最大的觀測值：50
  其 studentized residual：6.06
  其槓桿值：0.089
  其 Cook's D：0.329
```

	stud_resid	leverage	cooks_d
7	-0.328	0.713	0.024
9	-1.883	0.157	0.060
50	6.065	0.089	0.329
94	-1.457	0.310	0.087

> **參考答案**：門檻為 $h_i > 0.440$、$D_i > 0.053$。$|r_i| > 3$：50；高槓桿：7；$D_i > 4/n$：50, 9, 94。第 7 列的槓桿值 0.713 非常高，但 studentized residual 只有 -0.33、Cook's distance 僅 0.024（低於 4/n）：它的 x 很極端，但 y 落在迴歸平面附近，移除後係數幾乎不變，所以是**高槓桿但非影響點**。第 50 列的 studentized residual 為 6.06（離群值），槓桿值 0.089 不高，但 Cook's distance 0.329 是全體最大，是**影響點**。其他略高於 4/n 的觀測值只是輕微超標。

(d) 將 Cook's distance 大於 $4/n$ 的觀測值從訓練集移除（測試集不變），重新擬合 (b) 的模型。比較移除前後 V_1、V_2、V_3 的係數估計、R^2 與測試集 MSE，並說明你觀察到的變化。**以下 (e)–(g) 均使用移除後的訓練集。** (6%)

```
keep = diag.index[diag['cooks_d'] <= d_cut]
X_train2, y_train2 = X_train.loc[keep], y_train.loc[keep]
results2 = sm.OLS(y_train2, design.transform(X_train2)).fit()
mse_before = mean_squared_error(y_test, results.predict(design.transform(X_test)))
mse_after = mean_squared_error(y_test, results2.predict(design.transform(X_test)))
comp = pd.DataFrame({'before': results.params[['V1', 'V2', 'V3']], 'after': results2.params
[['V1', 'V2', 'V3']]})
comp.loc['R2'] = [results.rsquared, results2.rsquared]
comp.loc['test MSE'] = [mse_before, mse_after]
R('d_nrm', len(X_train) - len(keep)); R('d_R2b', results.rsquared); R('d_R2a', results2.rsqu
ared)
```

```
R('d_mseb', mse_before, 2); R('d_msea', mse_after, 2)
comp.round(3)
```

```
移除的觀測值筆數：3
移除前 R²：0.821
移除後 R²：0.914
移除前測試 MSE：3.63
移除後測試 MSE：3.50
```

	before	after
V1	3.636	3.425
V2	-2.496	-2.727
V3	2.580	2.310
R2	0.821	0.914
test MSE	3.627	3.496

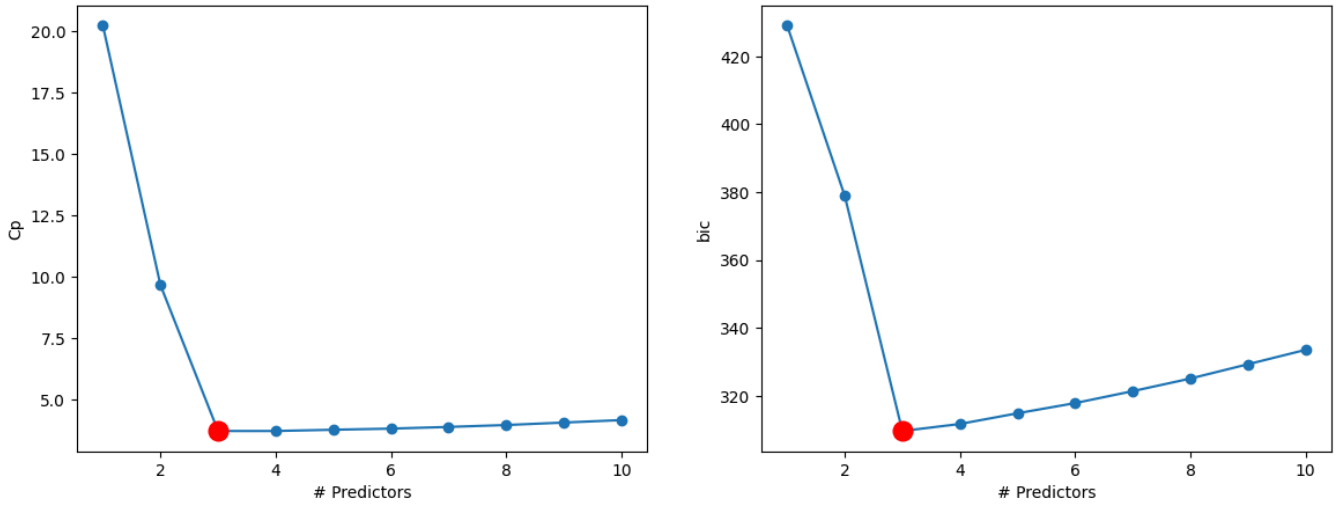
> **參考答案**：共移除 3 筆觀測值。R^2 由 0.821 提高到 0.914，測試 MSE 由 3.63 變為 3.50；V_1–V_3 的係數也有所改變（變動約 0.2–0.3，見上表），其中 R^2 的提升最明顯。影響點會把迴歸平面拉向自己，移除後殘差變異下降，係數更接近其餘資料所呈現的關係。但刪除觀測值必須有理由，不能只因為它不符合模型。

(e) 使用最佳子集選取法（best subset selection），對 $k = 1, \ldots, 10$ 找出 RSS 最小的模型，再分別以 $C_p = \frac{1}{n}(RSS + 2d\hat{\sigma}^2)$（$d$ 為自變數個數，$\hat{\sigma}^2 = RSS_{\text{full}}/(n-p-1)$ 取自包含全部十個自變數的模型）與 BIC 選出最佳模型。兩個準則選出的變數是否相同？回報所選模型的測試集 MSE。 (8%)

```
rss, sub, reg, kft = [], [], [], []
for k in range(1, 11):
    best_score, best_reg, best_subset = get_models(k, X_train2, y_train2)
    rss.append(best_score); sub.append(best_subset); reg.append(best_reg); kft.append(k)
results_bs = pd.DataFrame({'kft': kft, 'rss': rss, 'reg': reg, 'sub': sub}).set_index('kft')
sigma2 = OLS(y_train2, add_constant(X_train2)).fit().scale
n2 = X_train2.shape[0]
results_bs['Cp'] = (results_bs['rss'] + 2 * results_bs.index * sigma2) / n2
results_bs['bic'] = results_bs.apply(lambda row: row['reg'].bic, axis=1)
results_bs[['rss', 'Cp', 'bic', 'sub']]
```

kft	rss	Cp	bic	sub
1	1449.598521	20.235924	429.051538	(V1,)
2	680.386688	9.655038	378.868800	(V1, V2)
3	245.028769	3.711012	309.612898	(V1, V2, V3)
4	237.644936	3.711070	311.686513	(V1, V2, V3, V10)
5	233.955389	3.762438	314.836578	(V1, V2, V3, V7, V9)
6	229.848896	3.808015	317.838244	(V1, V2, V3, V7, V9, V10)
7	227.425442	3.876968	321.351734	(V1, V2, V3, V4, V7, V9, V10)
8	225.646965	3.954878	325.063144	(V1, V2, V3, V4, V5, V7, V9, V10)
9	225.455447	4.054830	329.278674	(V1, V2, V3, V4, V5, V6, V7, V9, V10)
10	225.335044	4.155769	333.516879	(V1, V2, V3, V4, V5, V6, V7, V8, V9, V10)

```
fig, axes = subplots(1, 2, figsize=(14, 5))
for ax, col in zip(axes, ['Cp', 'bic']):
    ax.plot(results_bs.index, results_bs[col], '-o')
    ax.plot(results_bs[col].idxmin(), results_bs[col].min(), 'or', ms=12)
    ax.set_xlabel('# Predictors'); ax.set_ylabel(col)
```



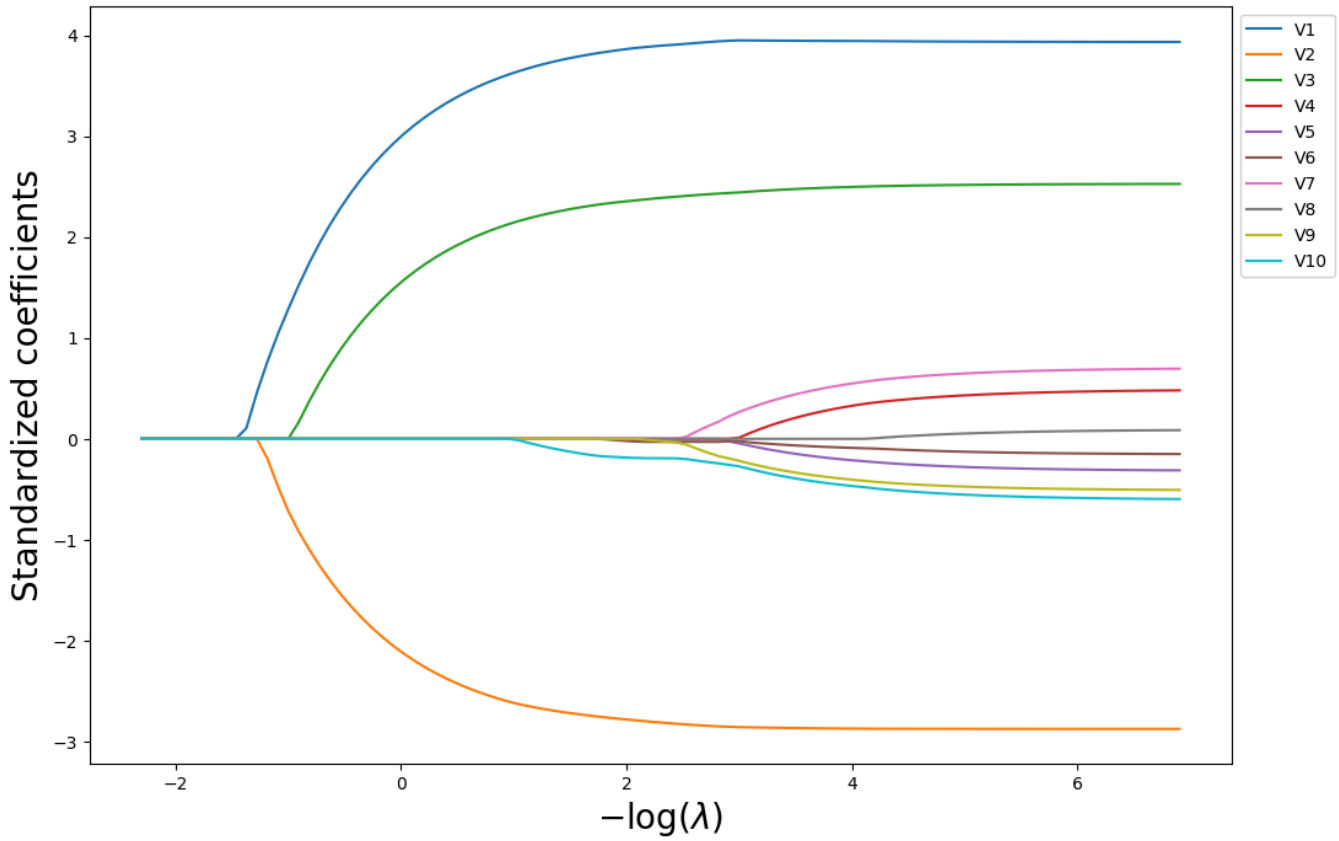

```
k_cp, k_bic = results_bs['Cp'].idxmin(), results_bs['bic'].idxmin()
sub_cp, sub_bic = list(results_bs.loc[k_cp, 'sub']), list(results_bs.loc[k_bic, 'sub'])
mse_cp = mean_squared_error(y_test, results_bs.loc[k_cp, 'reg'].predict(add_constant(X_test
[sub_cp])))
mse_bic = mean_squared_error(y_test, results_bs.loc[k_bic, 'reg'].predict(add_constant(X_tes
t[sub_bic])))
R('e_cp', sub_cp); R('e_bic', sub_bic); R('e_msecp', mse_cp, 2); R('e_msebic', mse_bic, 2)
R('e_cp3', results_bs.loc[3, 'Cp'], 4); R('e_cp4', results_bs.loc[4, 'Cp'], 4); R('e_sub4',
list(results_bs.loc[4, 'sub']))
R('e_bic3', results_bs.loc[3, 'bic'], 1); R('e_bic4', results_bs.loc[4, 'bic'], 1)
print('Cp :', sub_cp, round(mse_cp, 3)); print('BIC:', sub_bic, round(mse_bic, 3))
```

```
Cp 選出的變數：V1, V2, V3
BIC 選出的變數：V1, V2, V3
Cp 模型測試 MSE：2.79
BIC 模型測試 MSE：2.79
k=3 的 Cp：3.7110
k=4 的 Cp：3.7111
k=4 的最佳子集：V1, V2, V3, V10
k=3 的 BIC：309.6
k=4 的 BIC：311.7
Cp : ['V1', 'V2', 'V3'] 2.787
BIC: ['V1', 'V2', 'V3'] 2.787
```

> **參考答案**：C_p 選出 V1, V2, V3（測試 MSE 2.79）；BIC 選出 V1, V2, V3（測試 MSE 2.79）。但 C_p 在 $k=3$ 與 $k=4$ 幾乎打平（3.7110 vs 3.7111，$k=4$ 的模型為 V1, V2, V3, V10），$\hat{\sigma}^2$ 稍有不同就可能選出 4 個變數；BIC 則明確選出 3 個（309.6 vs 311.7）。BIC 對每多一個變數的懲罰（$\log n$）比 C_p / AIC 的 2 大，通常選出較小的模型。

(f) 在訓練集上擬合 Lasso regression，繪製十個自變數標準化後的迴歸係數（standardized coefficient estimates）隨 λ（sklearn 的 `alpha`）變化的路徑圖。λ 由 10^1 掃描到 10^{-3}，以對數刻度均勻取 100 個點；以 10 折交叉驗證（10-fold CV）選出使 CV 誤差最小的 λ。哪些係數被估計為 0？回報訓練集與測試集 MSE。 (8%)

```
alphas = 10**np.linspace(1, -3, 100)
scaler = StandardScaler()
Xs_train = scaler.fit_transform(X_train2)
lambdas, soln_array = skl.Lasso.path(Xs_train, y_train2, l1_ratio=1, alphas=alphas)[:2]   # l1_ratio=1 才是 lasso
soln_path = pd.DataFrame(soln_array.T, columns=X.columns, index=-np.log(lambdas))
path_fig, ax = subplots(figsize=(12, 8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel(r'$-\log(\lambda)$', fontsize=20); ax.set_ylabel('Standardized coefficients', fontsize=20)
ax.legend(loc='upper left', bbox_to_anchor=(1, 1));
```




```
kfold = skm.KFold(10, shuffle=True, random_state=2026)
lassoCV = skl.ElasticNetCV(alphas=alphas, l1_ratio=1, cv=kfold, max_iter=100000)
pipeCV = Pipeline(steps=[('scaler', StandardScaler()), ('lasso', lassoCV)])
pipeCV.fit(X_train2, y_train2)
tuned_lasso = pipeCV.named_steps['lasso']
coef = pd.Series(tuned_lasso.coef_, index=X.columns)
mse_tr = mean_squared_error(y_train2, pipeCV.predict(X_train2))
mse_te = mean_squared_error(y_test, pipeCV.predict(X_test))
R('f_alpha', tuned_lasso.alpha_, 4); R('f_zero', coef.index[coef == 0].tolist())
R('f_mse_tr', mse_tr, 2); R('f_mse_te', mse_te, 2)
print(tuned_lasso.alpha_); coef.round(4)
```

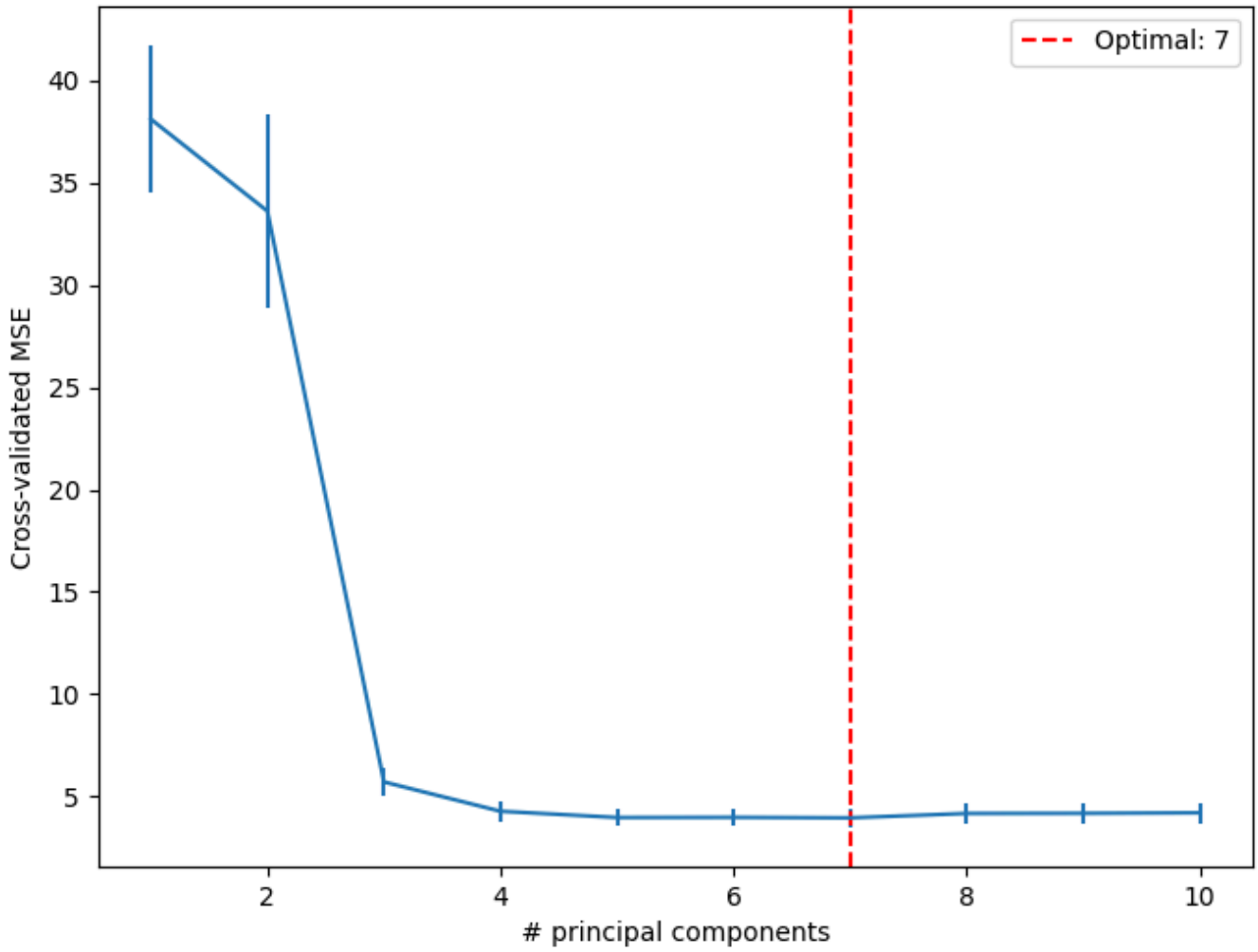
```
CV 選出的 λ：0.1262
係數為 0 的變數：V4, V5, V7, V8, V9
Lasso 訓練 MSE：3.36
Lasso 測試 MSE：2.75
0.12618568830660198
```

```
V1     3.8719
V2    -2.7882
V3     2.3616
V4    -0.0000
V5    -0.0000
V6    -0.0263
V7     0.0000
V8    -0.0000
V9    -0.0000
V10   -0.1894
dtype: float64
```

> **參考答案**：CV 選出的 $\lambda \approx 0.1262$。係數為 0 的變數：V4, V5, V7, V8, V9；V_1–V_3 保留，其餘相關區塊的變數不是被壓成 0 就是非常小。訓練 MSE 3.36，測試 MSE 2.75。

(g) 在訓練集上擬合主成分迴歸（PCR，Principal Components Regression），以 10 折交叉驗證從 $M = 1, \ldots, 10$ 中選擇主成分數 M，並繪製 CV MSE 對 M 的圖形。回報所選的 M 與測試集 MSE。為什麼 $M = 1$ 或 2 時 CV 誤差特別大？請搭配各主成分的解釋變異比例（explained variance ratio）說明。 (5%)

```
pipe = Pipeline([('scaler', StandardScaler()), ('pca', PCA()), ('linreg', LinearRegression
())])
param_grid = {'pca__n_components': range(1, 11)}
grid = skm.GridSearchCV(pipe, param_grid, cv=kfold, scoring='neg_mean_squared_error')
grid.fit(X_train2, y_train2)
n_comp = param_grid['pca__n_components']
pcr_fig, ax = subplots(figsize=(8, 6))
ax.errorbar(n_comp, -grid.cv_results_['mean_test_score'], grid.cv_results_['std_test_score']
/ np.sqrt(10))
ax.set_xlabel('# principal components'); ax.set_ylabel('Cross-validated MSE')
best_n_comp = n_comp[np.argmin(-grid.cv_results_['mean_test_score'])]
ax.axvline(best_n_comp, ls='--', c='r', label=f'Optimal: {best_n_comp}'); ax.legend();
```




```
evr = grid.best_estimator_.named_steps['pca'].explained_variance_ratio_
pca_full = PCA().fit(StandardScaler().fit_transform(X_train2))
mse_pcr = mean_squared_error(y_test, grid.best_estimator_.predict(X_test))
R('g_M', best_n_comp); R('g_mse', mse_pcr, 2); R('g_cv', np.round(-grid.cv_results_['mean_te
st_score'][:4], 1).tolist(), 1)
R('g_evr1', pca_full.explained_variance_ratio_[0], 2)
```

```
print('CV MSE:', np.round(-grid.cv_results_['mean_test_score'], 2))
print('explained variance ratio:', np.round(pca_full.explained_variance_ratio_, 3))
```

```
CV 選出的主成分數 M：7
PCR 測試 MSE：3.15
M=1–4 的 CV MSE：38.1, 33.6, 5.7, 4.3
第一主成分解釋變異比例：0.57
CV MSE: [38.1  33.61  5.72  4.28  3.98  3.99  3.96  4.18  4.18  4.21]
explained variance ratio: [0.567 0.125 0.111 0.084 0.041 0.018 0.017 0.015 0.014 0.01 ]
```

> **參考答案**：CV 選出 $M = 7$，測試 MSE 3.15。前四個 M 的 CV MSE 為 38.1, 33.6, 5.7, 4.3。第一主成分解釋了約 0.57 的變異，但它主要是 V_4–V_{10} 相關區塊的共同方向，與 y 無關；PCA 是非監督式的，只看 X 的變異、不看 y，所以前一兩個主成分沒有抓到 V_1–V_3 的資訊，要加到 $M \geq 3$、4 個成分後誤差才大幅下降，之後 CV MSE 大致持平。

題目二：使用 "Data2" 資料集進行分類分析

請使用 **"Data2"** 資料集，其中應變數 $y \in \{0, 1\}$（**類別 1 為正類（positive class）**），自變數為 $V_1, \ldots, V_{10}$。請以**測試集的 ROC AUC** 與**敏感度（sensitivity）**評估模型：

$$\text{sensitivity} = \frac{TP}{TP + FN}, \qquad \text{specificity} = \frac{TN}{TN + FP}$$

ROC AUC 請以預測機率計算；未特別說明時，分類門檻（threshold）為 0.5。

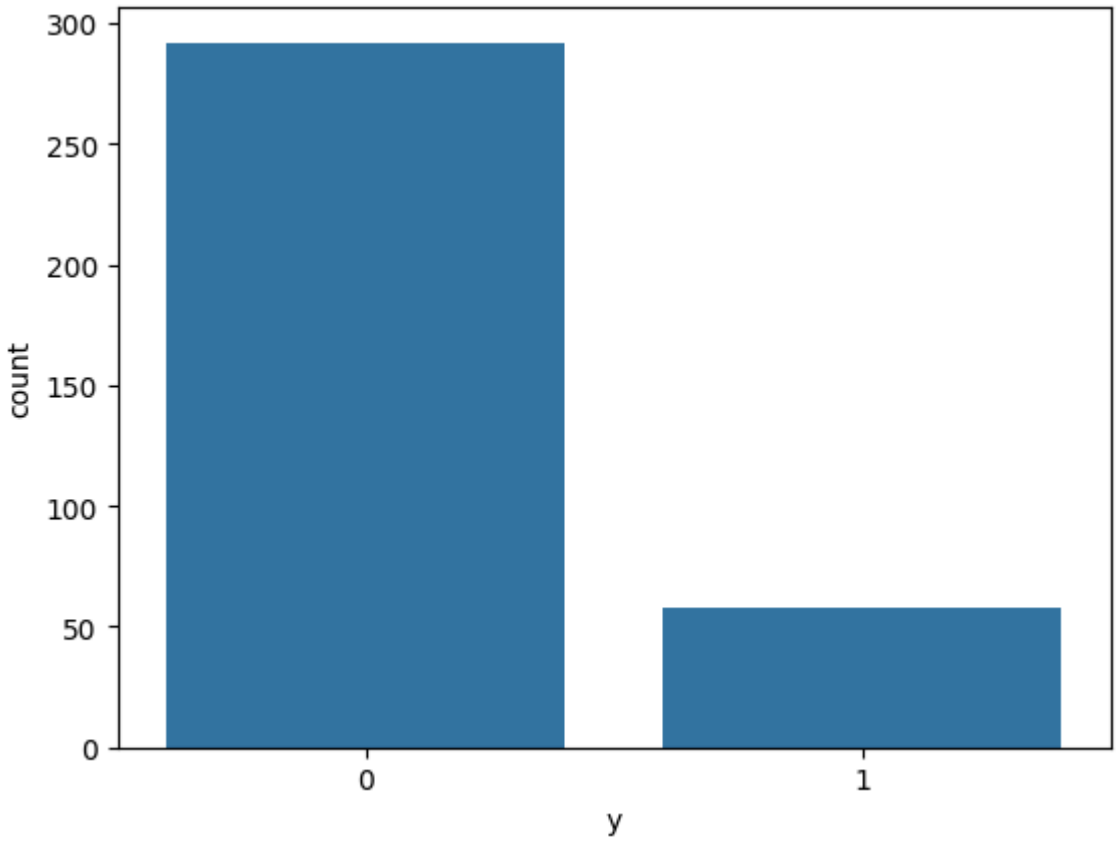
- 當使用具有隨機性的演算法（例如隨機抽樣）時，請設定 `random_state=2026` 以確保結果可重現。
- 若所選模型**不具尺度不變性（scale-invariance）**，請在建模前對輸入的自變數進行標準化（standardization）。

```
def report(y_true, probs, thr=0.5):
    labels = np.where(probs > thr, 1, 0)
    conf_mat = confusion_matrix(y_true, labels)   # True, predicted
    tn, fp, fn, tp = conf_mat[0, 0], conf_mat[0, 1], conf_mat[1, 0], conf_mat[1, 1]
    fpr, tpr, thresholds = roc_curve(y_true, probs)
    return {'AUC': auc(fpr, tpr), 'sensitivity': tp / (tp + fn),
            'specificity': tn / (tn + fp), 'accuracy': (tp + tn) / len(y_true)}
```

(a) 使用 `train_test_split` 以 `test_size=0.3` 將資料分為訓練集與測試集。分別統計訓練集與測試集中類別 0 與類別 1 的樣本數，並以 `countplot` 繪出訓練集的類別分布。若一個分類器把所有測試資料都預測為多數類別，其測試正確率（accuracy）是多少？ (5%)

```
df2 = pd.read_csv('Data2.csv')
X2 = df2.drop(columns='y'); y2 = df2['y']
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.3, random_state=
2026)
print(y2_train.value_counts(), y2_test.value_counts())
sns.countplot(x=y2_train);
```

```
y
0    292
1     58
Name: count, dtype: int64 y
0    130
1     20
Name: count, dtype: int64
```




```
base_acc = np.mean(y2_test == 0)       # 全部預測為多數類別 0
R('a2_tr', y2_train.value_counts().sort_index().tolist()); R('a2_te', y2_test.value_counts
().sort_index().tolist())
R('a2_base', base_acc)
base_acc
```

```
訓練集類別 0／1 筆數：292, 58
測試集類別 0／1 筆數：130, 20
全部預測為 0 的測試正確率：0.867
```

```
0.8666666666666667
```

> **參考答案**：訓練集類別 0 / 1 為 292, 58，測試集為 130, 20，資料明顯不平衡。全部預測為類別 0 的測試正確率就有 0.867，因此正確率不是好的評估指標。

(b) 使用訓練集建立邏輯斯迴歸模型（logistic regression，不加懲罰項）。回報測試集的 ROC AUC，並以門檻 0.5 列出測試集的混淆矩陣（confusion matrix）、敏感度與特異度（specificity）。 (8%)

```
design2 = MS(X2.columns).fit(X2_train)
glm = sm.GLM(y2_train, design2.transform(X2_train), family=sm.families.Binomial())
results_glm = glm.fit()
probs_glm = results_glm.predict(design2.transform(X2_test))
```

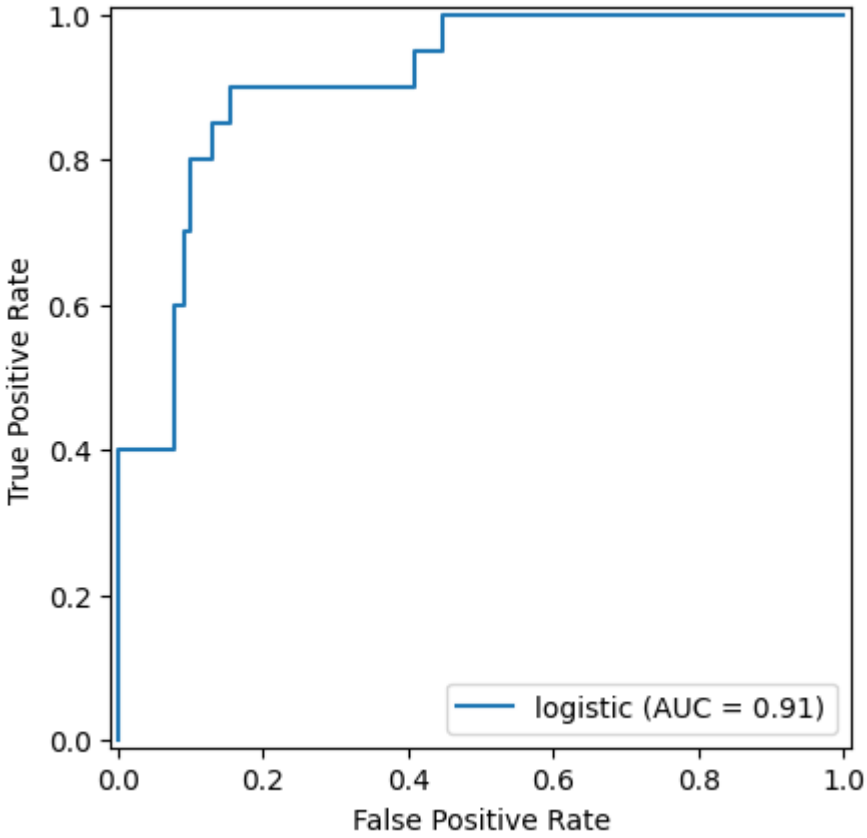
```
labels = np.where(probs_glm > 0.5, 1, 0)
confusion_table(labels, y2_test)
```

Truth	0	1
Predicted		
0	121	12
1	9	8

```
fpr, tpr, thresholds = roc_curve(y2_test, probs_glm)
RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=auc(fpr, tpr), estimator_name='logistic').plot();
rb = report(y2_test, probs_glm)
R('b2_auc', rb['AUC']); R('b2_sens', rb['sensitivity']); R('b2_spec', rb['specificity']); R
('b2_acc', rb['accuracy'])
rb
```

```
logistic 測試 AUC：0.908
logistic 敏感度 (門檻 0.5)：0.400
logistic 特異度 (門檻 0.5)：0.931
logistic 正確率 (門檻 0.5)：0.860
```

```
{'AUC': 0.9084615384615385,
 'sensitivity': 0.4,
 'specificity': 0.9307692307692308,
 'accuracy': 0.86}
```



> **參考答案**：測試 ROC AUC = 0.908。門檻 0.5 時敏感度只有 0.400、特異度 0.931（正確率 0.860，甚至略低於「全部猜 0」的 0.867）：模型的排序能力不錯，但 0.5 的門檻讓大部分正類被判成 0。

(c) 將 (b) 的分類門檻由 0.5 調整為 0.25，重新計算測試集的混淆矩陣、敏感度、特異度與正確率。與 (b) 相比有什麼變化？ROC AUC 是否會因此改變？為什麼在不平衡資料中只看正確率可能誤導？ (7%)

```
labels25 = np.where(probs_glm > 0.25, 1, 0)
print(confusion_table(labels25, y2_test))
rc = report(y2_test, probs_glm, 0.25)
R('c2_sens', rc['sensitivity']); R('c2_spec', rc['specificity']); R('c2_acc', rc['accurac
y'])
rc
```

```
Truth        0   1
Predicted
0          115   4
1           15  16
敏感度 (門檻 0.25)：0.800
特異度 (門檻 0.25)：0.885
正確率 (門檻 0.25)：0.873
```

```
{'AUC': 0.9084615384615385,
 'sensitivity': 0.8,
 'specificity': 0.8846153846153846,
 'accuracy': 0.8733333333333333}
```

> **參考答案**：門檻降到 0.25 後，敏感度由 0.400 提高到 0.800，特異度由 0.931 降到 0.885，正確率為 0.873：少漏判許多正類，代價是多一些誤報。ROC AUC 不變，因為 AUC 是對**所有**門檻整條 ROC 曲線的摘要，只取決於預測機率的排序。不平衡資料中，正確率主要由多數類決定，一個完全找不到正類的模型也能有很高的正確率。

(d) 使用訓練集分別建立線性判別分析（LDA）與二次判別分析（QDA）分類器，回報測試集的 ROC AUC 與（門檻 0.5，即預設 `predict`）敏感度。 (8%)

```
lda = LDA(store_covariance=True).fit(X2_train, y2_train)
qda = QDA(store_covariance=True).fit(X2_train, y2_train)
rl = report(y2_test, lda.predict_proba(X2_test)[:, 1])
rq = report(y2_test, qda.predict_proba(X2_test)[:, 1])
print(confusion_table(lda.predict(X2_test), y2_test)); print(confusion_table(qda.predict(X2_
test), y2_test))
R('d2_lda_auc', rl['AUC']); R('d2_lda_sens', rl['sensitivity']); R('d2_qda_auc', rq['AUC']);
R('d2_qda_sens', rq['sensitivity'])
pd.DataFrame({'LDA': rl, 'QDA': rq}).round(3)
```

```
Truth        0   1
Predicted
0          124  12
1            6   8
Truth        0   1
Predicted
0          126  17
1            4   3
LDA 測試 AUC：0.905
LDA 敏感度：0.400
QDA 測試 AUC：0.840
QDA 敏感度：0.150
```

	LDA	QDA
AUC	0.905	0.840
sensitivity	0.400	0.150
specificity	0.954	0.969
accuracy	0.880	0.860

> **參考答案**：LDA：AUC 0.905、敏感度 0.400；QDA：AUC 0.840、敏感度 0.150。QDA 要為少數類（訓練集僅約 58 筆）估計一個 10×10 的共變異數矩陣，變異大，表現不如 LDA。

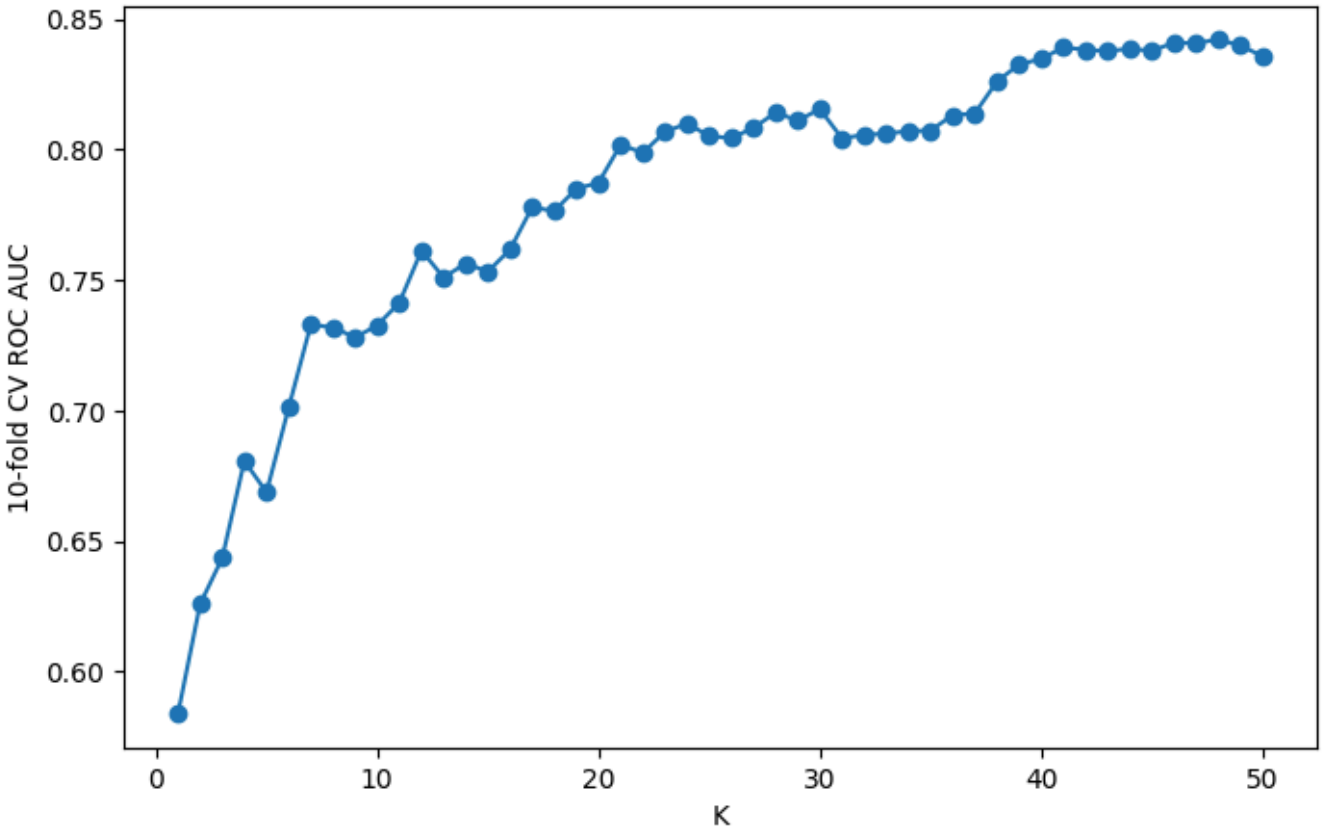
(e) 使用標準化後的自變數建立 KNN 分類器，以 10 折交叉驗證、ROC AUC 為準則，從 $K = 1, \ldots, 50$ 中選出最佳 K。回報所選的 K、測試集 ROC AUC 與門檻 0.5 的敏感度。你觀察到什麼現象？ (8%)

```
scaler2 = StandardScaler().fit(X2_train)
Z_train, Z_test = scaler2.transform(X2_train), scaler2.transform(X2_test)
kfold10 = skm.KFold(10, shuffle=True, random_state=2026)
grid_knn = skm.GridSearchCV(KNeighborsClassifier(), {'n_neighbors': range(1, 51)}, cv=kfold1
0, scoring='roc_auc')
grid_knn.fit(Z_train, y2_train)
K = grid_knn.best_params_['n_neighbors']
knn = grid_knn.best_estimator_
rk = report(y2_test, knn.predict_proba(Z_test)[:, 1])
print(confusion_table(knn.predict(Z_test), y2_test))
R('e2_K', K); R('e2_auc', rk['AUC']); R('e2_sens', rk['sensitivity'])
K, rk
```

```
Truth        0   1
Predicted
0          130  20
1            0   0
CV 選出的 K：48
KNN 測試 AUC：0.908
KNN 敏感度 (門檻 0.5)：0.000
```

```
(48,
 {'AUC': 0.9080769230769231,
  'sensitivity': 0.0,
  'specificity': 1.0,
  'accuracy': 0.8666666666666667})
```

```
fig, ax = subplots(figsize=(8, 5))
ax.plot(range(1, 51), grid_knn.cv_results_['mean_test_score'], '-o')
ax.set_xlabel('K'); ax.set_ylabel('10-fold CV ROC AUC');
```



> **參考答案**：CV 選出 $K = 48$，測試 AUC 0.908，但門檻 0.5 的敏感度為 0.000。K 很大時，鄰居中正類比例很少超過 0.5，模型幾乎把所有觀測值判為多數類；AUC 只看排序所以仍然不錯。這再次說明門檻的選擇與 AUC 是兩件事。

(f) 綜合 (b)–(e) 的結果，若應用上較重視找出正類（減少漏判），你會建議使用哪一個模型與門檻？請說明理由。 (4%)

> **參考答案**：建議使用邏輯斯迴歸並把門檻降到約 0.25：其 AUC（0.908）在各模型中最高之列，降低門檻後敏感度由 0.400 升至 0.800。LDA 的 AUC 相近，可列為候選，但本題只對邏輯斯迴歸測試了 0.25 的門檻；QDA 的 AUC 較低，KNN 在 0.5 門檻下完全找不到正類。最終門檻應在驗證資料上依漏判與誤報的成本決定，而不是用測試集挑選。

題目三：Bootstrap 估計標準誤

本部分延續題目一的 **"Data1"** 資料與訓練集切分（`test_size=0.25`，`random_state=2026`）。

(a) 題目一 (b) 以 statsmodels 擬合的多元線性迴歸模型（`results = sm.OLS(...).fit()`），其 `results.summary()` 報告的迴歸係數標準誤（standard error）是根據哪些模型假設推導出來的公式？Bootstrap 估計標準誤的基本想法是什麼？為什麼當資料中存在影響點時，兩者可能明顯不同？ (8%)

> **參考答案**：以講義的簡單線性迴歸為例：$SE(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^{n}(x_i-\bar{x})^2}$，$SE(\hat{\beta}_0)^2 = \sigma^2\left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^{n}(x_i-\bar{x})^2}\right]$，其中 $\sigma^2 = \mathrm{Var}(\varepsilon)$ 以 $RSE^2 = RSS/(n-p-1)$ 估計（簡單迴歸為 $n-2$）；多元迴歸的標準誤公式也是同樣的想法。這些公式假設線性模型正確、誤差獨立、同變異數（homoscedasticity），並以單一的 $\hat{\sigma}^2$ 代表所有觀測值的雜訊大小。Bootstrap 把觀測到的樣本當作母體，以**放回抽樣**（sampling with replacement）重複抽出大小為 n 的樣本、重新估計係數，用這些估計值的標準差近似抽樣分配的標準差；它不必套用同變異數的 OLS 標準誤公式，但仍假設各列（觀測值）可視為來自同一母體的獨立樣本。若存在影響點，係數估計會隨「這一筆有沒有被抽到、抽到幾次」大幅變動，bootstrap SE 會反映這種不穩定；公式 SE 則把它平均進 $\hat{\sigma}^2$，兩者因此可能明顯不同。

(b) 使用題目一 (b) 的**原始訓練集**（未移除任何觀測值；若遇到索引錯誤，可先 `reset_index(drop=True)`），以 bootstrap（$B = 1000$，亂數種子 2026）估計全模型中 V_1、V_2、V_3 係數的標準誤，並與題目一 (b) 模型 `results.summary()` 報告的標準誤比較。接著對題目一 (d) **移除影響點後**的訓練集重做一次。請將四組標準誤整理成表格並解釋差異。(12%)

```
D_orig = X_train.copy()                          # 題目一 (b) 的原始訓練集
D_orig['y'] = y_train
D_orig = D_orig.reset_index(drop=True)
D_rm = X_train2.copy()                           # 題目一 (d) 移除後的訓練集
D_rm['y'] = y_train2
D_rm = D_rm.reset_index(drop=True)
full_func = partial(boot_OLS, MS(list(X.columns)), 'y')
se_boot_orig = boot_SE(full_func, D_orig, B=1000, seed=2026)
se_boot_rm = boot_SE(full_func, D_rm, B=1000, seed=2026)
```

```
print(results.summary())          # 題目一 (b) 原始訓練集的模型
print(results2.summary())         # 題目一 (d) 移除影響點後的模型
```

OLS Regression Results

Dep. Variable:	y	R-squared:	0.821
Model:	OLS	Adj. R-squared:	0.793
Method:	Least Squares	F-statistic:	29.35
Date:	Tue, 20 Oct 2026	Prob (F-statistic):	3.39e-20
Time:	20:53:15	Log-Likelihood:	-184.29
No. Observations:	75	AIC:	390.6
Df Residuals:	64	BIC:	416.1
Df Model:	10		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
intercept	1.4201	0.384	3.700	0.000	0.653	2.187
V1	3.6361	0.330	11.017	0.000	2.977	4.295
V2	-2.4957	0.362	-6.894	0.000	-3.219	-1.772
V3	2.5796	0.346	7.465	0.000	1.889	3.270
V4	0.0364	0.790	0.046	0.963	-1.541	1.614
V5	-0.4071	0.765	-0.532	0.596	-1.935	1.120
V6	0.2028	0.877	0.231	0.818	-1.550	1.955
V7	0.5587	0.775	0.721	0.473	-0.989	2.106
V8	0.4063	0.804	0.505	0.615	-1.200	2.012
V9	-0.2961	0.739	-0.400	0.690	-1.773	1.181
V10	-0.6727	0.846	-0.795	0.430	-2.363	1.018

Omnibus:	81.597	Durbin-Watson:	2.063
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1151.612
Skew:	3.080	Prob(JB):	8.52e-251
Kurtosis:	21.181	Cond. No.	7.79

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

OLS Regression Results

Dep. Variable:	y	R-squared:	0.914
Model:	OLS	Adj. R-squared:	0.900
Method:	Least Squares	F-statistic:	65.11
Date:	Tue, 20 Oct 2026	Prob (F-statistic):	9.84e-29
Time:	20:53:15	Log-Likelihood:	-143.24
No. Observations:	72	AIC:	308.5
Df Residuals:	61	BIC:	333.5
Df Model:	10		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
intercept	1.2505	0.247	5.072	0.000	0.758	1.744
V1	3.4255	0.216	15.824	0.000	2.993	3.858
V2	-2.7270	0.235	-11.608	0.000	-3.197	-2.257
V3	2.3100	0.226	10.244	0.000	1.859	2.761
V4	0.4465	0.504	0.885	0.379	-0.562	1.455
V5	-0.3437	0.514	-0.669	0.506	-1.371	0.684
V6	-0.1517	0.564	-0.269	0.789	-1.280	0.977
V7	0.6843	0.489	1.399	0.167	-0.294	1.662
V8	0.0933	0.517	0.181	0.857	-0.940	1.126
V9	-0.5103	0.500	-1.021	0.311	-1.509	0.489
V10	-0.5741	0.534	-1.076	0.286	-1.641	0.493

```
Omnibus:                        0.657   Durbin-Watson:                   2.002
Prob(Omnibus):                  0.720   Jarque-Bera (JB):                0.527
Skew:                          -0.207   Prob(JB):                        0.769
Kurtosis:                       2.938   Cond. No.                         7.69
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
se_formula_orig = results.bse    # 即 summary() 係數表中的 std err 欄
se_formula_rm = results2.bse
tab = pd.DataFrame({'boot (original)': se_boot_orig, 'formula (original)': se_formula_orig,
                    'boot (removed)': se_boot_rm, 'formula (removed)': se_formula_rm}).loc
[['V1', 'V2', 'V3']]
R('p3_bo', tab['boot (original)'].round(3).tolist()); R('p3_fo', tab['formula (original)'].r
ound(3).tolist())
R('p3_br', tab['boot (removed)'].round(3).tolist()); R('p3_fr', tab['formula (removed)'].rou
nd(3).tolist())
R('p3_ratio', (tab['boot (original)'] / tab['formula (original)']).mean(), 2)
tab.round(3)
```

```
原始訓練集 bootstrap SE (V1, V2, V3)：0.389, 0.449, 0.445
原始訓練集公式 SE (V1, V2, V3)：0.330, 0.362, 0.346
移除後 bootstrap SE (V1, V2, V3)：0.211, 0.235, 0.198
移除後公式 SE (V1, V2, V3)：0.216, 0.235, 0.226
原始訓練集 bootstrap SE／公式 SE 平均倍數：1.24
```

	boot (original)	formula (original)	boot (removed)	formula (removed)
V1	0.389	0.330	0.211	0.216
V2	0.449	0.362	0.235	0.235
V3	0.445	0.346	0.198	0.226

> **參考答案**：原始訓練集：bootstrap SE 0.389, 0.449, 0.445，公式 SE 0.330, 0.362, 0.346，bootstrap 平均約為公式的 1.24 倍（高約兩成）。移除影響點後：bootstrap SE 0.211, 0.235, 0.198，公式 SE 0.216, 0.235, 0.226，兩者相當接近。有影響點時，bootstrap 樣本是否包含那一筆會讓係數明顯改變，bootstrap SE 因此較大，反映了重抽樣下係數估計的變動，這與公式 SE 所依據的模型式估計不同；移除後兩種 SE 較一致，但這並不等於驗證了模型假設。